

SOFTWARE ENGINEERING

Complete Interview & Placement Notes

SDLC • Design Patterns • Architecture • Agile • Testing • UML

Covers all key topics asked in campus placements, technical interviews & GATE

1. Software Development Life Cycle (SDLC)

Phases, Models & Key Concepts

What is SDLC?

SDLC is a structured process for planning, creating, testing, and delivering software. It provides a framework that defines tasks to be performed at each step.

SDLC Phases

Phase	Activities / Output
Planning	Feasibility study, resource estimation, project scheduling
Requirements	SRS (Software Requirements Specification) document
System Design	HLD (High-Level Design), LLD (Low-Level Design)
Implementation	Coding, unit testing by developers
Testing	Integration, system, acceptance testing
Deployment	Release to production environment
Maintenance	Bug fixes, enhancements, performance tuning

SDLC Models

Waterfall Model

- Sequential, phase-by-phase approach.
- Each phase must be completed before the next begins.
- Best for: well-defined, stable requirements.
- Disadvantage: no working software until late; inflexible to changes.

V-Model (Validation & Verification)

- Extension of Waterfall; each dev phase has a corresponding test phase.
- Testing planned in parallel with development.
- Best for: projects needing high reliability (medical, aerospace).

Iterative Model

- Development done in repeated cycles (iterations).
- Each iteration produces a partial but working version.
- Requirements may evolve over iterations.

Spiral Model

- Combines Waterfall + Prototyping with risk analysis at each loop.
- 4 phases: Planning → Risk Analysis → Engineering → Evaluation.
- Best for: large, high-risk, complex projects.

Agile Model

- Iterative & incremental; working software delivered in sprints (2–4 weeks).
- Customer collaboration over contract negotiation.
- Frameworks: Scrum, Kanban, XP, SAFe.

RAD (Rapid Application Development)

- Short development cycles using prototyping.
- Heavy user involvement; best for time-constrained projects.

★ *Interview Tip: Know pros & cons of each model. Waterfall vs Agile is a very common question!*

Feasibility Study Types

- **Technical** – Can we build it with current technology?
- **Economic** – Is it cost-effective? (Cost-Benefit Analysis)
- **Operational** – Will users accept and use the system?
- **Schedule** – Can it be delivered on time?
- **Legal** – Does it comply with laws/regulations?

2. Requirements Engineering

SRS, Types of Requirements & Analysis

Types of Requirements

Type	Description & Examples
Functional	What the system should do — Login, Search, Payment processing
Non-Functional	How the system behaves — Performance, Security, Scalability, Reliability
Domain	Requirements from the application domain — e.g., banking rules
User Requirements	High-level statements for end users (natural language)
System Requirements	Detailed technical specifications for developers

SRS Document (IEEE 830)

- Introduction: Purpose, scope, definitions, overview
- Overall Description: Product perspective, functions, user characteristics
- Specific Requirements: Functional, non-functional, external interface requirements
- Appendices & Index

■ SRS qualities: Correct, Unambiguous, Complete, Consistent, Ranked, Verifiable, Modifiable, Traceable

Requirement Elicitation Techniques

- Interviews – One-on-one with stakeholders
- Questionnaires – Surveys for large groups
- Observation – Watching users work (ethnography)
- Prototyping – Build a rough model to clarify requirements
- Use Case Analysis – Identify actors and interactions
- Brainstorming – Group idea generation
- JAD Sessions – Joint Application Development workshops

3. Software Design Principles

SOLID, Coupling, Cohesion, Design Patterns

SOLID Principles

Principle	Meaning
S – Single Responsibility	A class should have only ONE reason to change
O – Open/Closed	Open for extension, CLOSED for modification
L – Liskov Substitution	Subtypes must be substitutable for their base types
I – Interface Segregation	Many specific interfaces > one general interface
D – Dependency Inversion	Depend on abstractions, NOT on concretions

Coupling & Cohesion

Coupling: Degree of interdependence between modules. **Low coupling = good design.**

Coupling Type	Description
Content	One module modifies another's internal data (worst)
Common	Modules share global data
Control	One module controls logic of another
Stamp	Modules share composite data structure
Data	Modules share data via parameters (best)

Cohesion: Degree to which elements of a module belong together. **High cohesion = good design.**

- Functional > Sequential > Communicational > Procedural > Temporal > Logical > Coincidental (worst)

Design Patterns (GoF — 23 Patterns)

Category	Patterns & One-liner
Creational (Object creation)	Singleton (one instance), Factory Method (subclass decides), Abstract Factory (family of objects), Builder (step-by-step), Prototype (clone)
Structural (Class composition)	Adapter (interface converter), Bridge (abstraction-impl split), Composite (tree), Decorator (add behavior), Facade (simplified interface), Flyweight (share objects), Proxy (surrogate)
Behavioral (Communication)	Observer (publish-subscribe), Strategy (interchangeable algorithms), Command (encapsulate request), Iterator, Template Method, State, Chain of Responsibility, Mediator, Memento, Visitor, Interpreter

★ Most asked: Singleton, Factory, Observer, Strategy, Decorator, Facade, Proxy

Architecture Patterns

- **MVC** – Model-View-Controller: separates data, UI, and logic
- **MVP** – Model-View-Presenter: Presenter handles all UI logic
- **MVVM** – Model-View-ViewModel: two-way data binding (Angular, WPF)

- **Layered (N-Tier)** – Presentation → Business Logic → Data Access
- **Microservices** – Small, independently deployable services
- **Event-Driven** – Components communicate via events/messages
- **SOA** – Service-Oriented Architecture; services via enterprise bus
- **Hexagonal (Ports & Adapters)** – Core logic isolated from external systems

4. UML — Unified Modeling Language

14 Diagram Types, Relationships & Notations

Diagram Type	Purpose & Key Elements
Use Case	Shows system functionality from user perspective. Actors, Use Cases, <>, <>
Class Diagram	Static structure: Classes, attributes, methods, relationships (association, aggregation, composition, inheritance)
Sequence	Object interactions over time; messages between lifelines
Activity	Workflow / flowchart; actions, decisions, forks, joins
State Machine	States an object goes through; events & transitions
Component	Physical components and their interfaces
Deployment	Hardware nodes and software deployment
Object Diagram	Snapshot of objects at a point in time
Package Diagram	Grouping of related classes into packages
Communication	Similar to sequence but emphasizes links between objects
Timing Diagram	Shows state changes over real time
Interaction Overview	Combines activity + sequence diagrams

Class Diagram Relationships

Relationship	Notation & Meaning
Association	Solid line – general relationship between classes
Aggregation	Hollow diamond – 'has-a' (child can exist independently)
Composition	Filled diamond – strong 'has-a' (child cannot exist alone)
Inheritance	Hollow triangle arrow – 'is-a' relationship
Realization	Dashed + hollow triangle – class implements interface
Dependency	Dashed arrow – uses another class temporarily

■ Draw a Use Case + Class + Sequence diagram for common scenarios like 'ATM', 'Library System', 'E-Commerce'

5. Agile Methodology & Scrum

Manifesto, Scrum Roles, Events, Artifacts

Agile Manifesto — 4 Values

- Individuals and interactions OVER processes and tools
- Working software OVER comprehensive documentation
- Customer collaboration OVER contract negotiation
- Responding to change OVER following a plan

Scrum Framework

Element	Details
Roles	Product Owner (PO), Scrum Master (SM), Development Team
Sprint	Time-boxed iteration of 1–4 weeks; produces a potentially shippable increment
Artifacts	Product Backlog (all features), Sprint Backlog (sprint tasks), Increment (done work)
Events	Sprint Planning, Daily Scrum (15 min standup), Sprint Review, Sprint Retrospective
DoD	Definition of Done: agreed criteria a story must meet to be 'done'
Velocity	Story points completed per sprint; used for forecasting

Kanban vs Scrum

Kanban	Scrum
Continuous flow	Fixed-length sprints
No roles defined	Defined roles (PO, SM, Team)
WIP limits	Sprint backlog commitment
Pull system	Push + Pull
Board visualization	Scrum board + ceremonies

Agile Estimation Techniques

- **Story Points** – Relative effort using Fibonacci (1, 2, 3, 5, 8, 13...)
- **Planning Poker** – Team members simultaneously reveal cards, discuss differences
- **T-Shirt Sizing** – XS, S, M, L, XL categories for quick estimates
- **Bucket System** – Items sorted into buckets of complexity

★ *User Story format: 'As a [user], I want [feature] so that [benefit]'. Acceptance criteria define when it's done.*

6. Software Testing

Types, Levels, Techniques & Metrics

Levels of Testing

Level	Done By / Purpose
Unit Testing	Developer; tests individual functions/methods in isolation
Integration Testing	Testers; tests interaction between combined modules
System Testing	QA team; end-to-end testing on complete system
Acceptance Testing	Client/Users; UAT – validates against business requirements
Regression Testing	After changes; ensures existing features still work
Smoke Testing	Quick sanity check; is the build stable enough to test?
Sanity Testing	Narrow, focused check after a specific bug fix
Performance Testing	Load, Stress, Spike, Soak testing for non-functional requirements

Black Box vs White Box vs Grey Box

Type	Tester Knows	Techniques
Black Box	Nothing about internals	Equivalence Partitioning, Boundary Value Analysis, Decision Table
White Box	Full source code access	Statement Coverage, Branch Coverage, Path Coverage
Grey Box	Partial knowledge	Combination of both; database testing

Test Coverage Criteria

- **Statement Coverage** – Every statement executed at least once
- **Branch Coverage** – Every branch (true/false) covered
- **Path Coverage** – Every possible path through code
- **Condition Coverage** – Every boolean sub-expression evaluated T and F

Equivalence Partitioning & Boundary Value Analysis

Equivalence Partitioning (EP): Divide input into valid and invalid classes. Test one value from each class. E.g., age 18–60 valid: test 30 (valid), 5 (invalid), 70 (invalid).

Boundary Value Analysis (BVA): Test at boundaries — min, min+1, max-1, max, and just outside boundaries. E.g., for 1–100: test 0, 1, 2, 99, 100, 101.

Key Testing Terms

Term	Definition
Error/Mistake	Human action that produces incorrect result
Defect/Bug	Flaw in software that causes incorrect behavior

Term	Definition
Failure	System deviates from expected behavior at runtime
Test Case	Set of conditions + expected result for testing
Test Plan	Document describing scope, approach, resources, schedule
Test Suite	Collection of related test cases
Alpha Testing	Testing by internal users at developer's site
Beta Testing	Testing by external users in real environment
Mutation Testing	Small changes (mutations) injected to verify test quality

★ V-Model maps each dev phase to a test phase: Requirements→AT, Design→ST, Detailed Design→IT, Coding→UT

7. Software Metrics & Project Management

Estimation, Metrics, Risk & Quality

Size & Effort Estimation

Technique	Description
LOC (Lines of Code)	Simple count; machine/language dependent
Function Points (FP)	Measures functionality: inputs, outputs, queries, files, interfaces
Use Case Points	Derived from use case complexity and technical factors
COCOMO	Constructive Cost Model: $\text{Effort} = a \times (\text{KLOC})^b$; Basic / Intermediate / Detailed
Planning Poker	Agile estimation using story points (team-based)
Delphi Technique	Expert opinion consensus through iterative rounds

COCOMO Model

- **Basic COCOMO:** $\text{Effort} = a(\text{KLOC})^b$; $\text{Duration} = c(\text{Effort})^d$
- **Organic:** Small teams, familiar domain. $a=2.4$, $b=1.05$
- **Semi-detached:** Mixed experience. $a=3.0$, $b=1.12$
- **Embedded:** Tight constraints. $a=3.6$, $b=1.20$

Key Software Metrics

Metric	Formula / Meaning
Cyclomatic Complexity	$M = E - N + 2P$ (edges–nodes+2×components); measures code complexity
Defect Density	Defects / KLOC; lower = better quality
MTBF	Mean Time Between Failures = $\text{MTTF} + \text{MTTR}$
MTTF	Mean Time To Failure: avg time system works before failure
MTTR	Mean Time To Repair: avg time to fix a failure
Availability	$\text{MTTF} / (\text{MTTF} + \text{MTTR}) \times 100\%$
Test Coverage	% of code/branches/paths tested
Velocity	Story points completed per sprint (Agile)

Risk Management

- **Risk Identification** – List all potential risks
- **Risk Analysis** – $\text{Probability} \times \text{Impact} = \text{Risk Exposure}$
- **Risk Planning** – Mitigation, avoidance, transfer, acceptance strategies
- **Risk Monitoring** – Continuous tracking throughout project

Software Quality Attributes (ISO 25010)

- **Functionality** – Does it do what it should?

- **Reliability** – Does it perform consistently?
- **Usability** – Is it easy to use?
- **Efficiency/Performance** – Does it respond fast with minimal resources?
- **Maintainability** – Easy to modify and fix?
- **Portability** – Can it run on different environments?
- **Security** – Protected against unauthorized access?

8. SCM, Version Control & DevOps

Git, CI/CD, Configuration Management

Software Configuration Management (SCM)

- **Version Control** – Track changes to code (Git, SVN)
- **Change Management** – Process for requesting/approving changes
- **Build Management** – Automated build (Maven, Gradle, Make)
- **Release Management** – Planning and managing software releases
- **Configuration Audit** – Verify config items meet requirements

Git Concepts

Concept	Description
Repository	Storage for code + full history
Commit	Snapshot of changes with message
Branch	Independent line of development
Merge	Combine changes from two branches
Rebase	Move/combine commits onto another base; cleaner history
Pull Request	Request to merge feature branch → main (code review)
Conflict	Occurs when same lines changed in two branches
git stash	Temporarily save uncommitted changes
git cherry-pick	Apply a specific commit from another branch
git bisect	Binary search to find the commit that introduced a bug

CI/CD Pipeline

- **Continuous Integration (CI)**: Automatically build & test code on every commit
- **Continuous Delivery (CD)**: Automatically prepare release to staging; manual deploy to prod
- **Continuous Deployment**: Fully automated release to production
- Tools: Jenkins, GitHub Actions, GitLab CI, CircleCI, Travis CI

■ Know the difference: Continuous Delivery = manual prod deploy; Continuous Deployment = fully automated

9. Software Maintenance, Reliability & Ethics

Types of Maintenance, Reliability Models

Types of Software Maintenance

Type	Description & Example
Corrective	Fix bugs discovered after release
Adaptive	Modify software to work in a new environment (OS upgrade, new hardware)
Perfective	Improve performance or add new features (user requests)
Preventive	Restructure/refactor code to prevent future problems (technical debt)

★ *Lehman's Laws: Software must be continuously adapted or it becomes progressively less satisfactory.*

Software Reliability

- Reliability: Probability that software performs its intended function correctly for a specified time
- Fault Tolerance: System continues to operate even when components fail
- Redundancy: Replication of critical components (hardware/software)
- Failure Rate (λ) = Number of failures / Total operating time
- Reliability $R(t) = e^{(-\lambda t)}$

Software Re-engineering

- **Reverse Engineering:** Extract design from existing code
- **Re-engineering:** Restructure existing system without changing functionality
- **Forward Engineering:** Normal development from requirements to code
- **Refactoring:** Improve internal code structure without changing external behavior

Software Ethics (ACM/IEEE Code)

- Public – Act in the public interest
- Client & Employer – Act in best interest of client and employer
- Product – Ensure software meets highest professional standards
- Judgment – Maintain integrity and independence in professional judgment
- Management – Promote ethical management of software development

10. Top Interview Questions & Quick Answers

Commonly Asked in Placements & Technical Rounds

Q1. What is the difference between verification and validation?

Verification: Are we building the product RIGHT? (reviews, walkthroughs) | Validation: Are we building the RIGHT product? (testing with actual requirements)

Q2. What is a software process model? Which one would you choose for a banking system?

A framework describing how software is developed. For banking (high reliability, stable requirements), V-Model or Waterfall with rigorous testing phases is preferred.

Q3. Explain Cohesion and Coupling with an example.

Cohesion = how well a module's elements belong together (high is good). Coupling = dependency between modules (low is good). E.g., a `PaymentProcessor` class that only processes payments = high cohesion. It shouldn't depend on `UI` = low coupling.

Q4. What is the difference between static and dynamic testing?

Static: Testing without executing code (code reviews, walkthroughs, inspections). Dynamic: Testing by executing the code (unit, integration, system testing).

Q5. What is a use case and how does it differ from a user story?

Use Case: Detailed description of system behavior with actors, preconditions, main flow, alternative flows. User Story: Short, informal description — 'As a [user], I want [goal] so that [reason]'. User stories are Agile-centric.

Q6. Explain the Singleton design pattern and where you'd use it.

Ensures only ONE instance of a class exists; provides a global access point. Use for: Database connections, Logger, Configuration manager, Thread pool.

Q7. What is technical debt?

Implied cost of rework when choosing quick/easy solutions over better approaches. Can accumulate over time; managed through refactoring, code reviews, and good practices.

Q8. What is the difference between Alpha and Beta testing?

Alpha: Done by internal team at developer's site before release. Beta: Done by actual end users in real environment before final release.

Q9. What is Cyclomatic Complexity?

Measures the number of linearly independent paths through code. Formula: $M = E - N + 2P$. Lower is simpler; > 10 indicates high complexity.

Q10. What are SOLID principles? Give an example of SRP.

5 OOP design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. SRP: A class 'Invoice' should only handle invoice data, NOT printing — that's a separate 'InvoicePrinter' class.

Q11. What is the difference between Agile and Scrum?

Agile is a philosophy/mindset with 4 values and 12 principles. Scrum is a specific Agile FRAMEWORK with defined roles (PO, SM, Dev Team), events (Sprint, Standup), and artifacts.

Q12. What is Function Point Analysis?

Size estimation technique based on functional complexity: counts External Inputs, Outputs, Inquiries, Internal Files, External Interfaces. Language-independent; useful for estimating effort and cost.

11. Quick Revision Cheat Sheet

Formulas, Acronyms & Must-Remember Facts

Key Formulas

Formula	Description
$M = E - N + 2P$	Cyclomatic Complexity
$Effort = a(KLOC)^b$	COCOMO Basic
$MTBF = MTTF + MTTR$	Mean Time Between Failures
$Availability = MTTF/MTBF$	System Availability ($\times 100\%$)
$R(t) = e^{(-\lambda t)}$	Reliability function
$Defect\ Density = D/KLOC$	Quality metric
$FP = UFP \times CAF$	Function Points (CAF = complexity adjustment)
$Risk\ Exposure = P \times I$	Probability $\times$ Impact

Acronyms You Must Know

Acronym	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
HLD	High-Level Design
LLD	Low-Level Design
SDLC	Software Development Life Cycle
SCM	Software Configuration Management
UAT	User Acceptance Testing
TDD	Test-Driven Development
BDD	Behavior-Driven Development
DDD	Domain-Driven Design
CI/CD	Continuous Integration / Continuous Delivery
KLOC	Kilo Lines of Code (1000 LOC)
FP	Function Points
MTBF	Mean Time Between Failures
COCOMO	Constructive Cost Model

Last-Minute Power Tips

■ Waterfall: requirements fixed. Agile: requirements can change. Spiral: high-risk projects.

■ Coupling: lower is better. Cohesion: higher is better. Remember this always!

■ V-Model = Waterfall + testing at each phase. Best for safety-critical systems.

■ Singleton, Factory, Observer, Strategy are the 4 most frequently asked design patterns.

■ Unit test → Integration test → System test → Acceptance test (bottom-up pyramid).

■ Verification = process quality; Validation = product quality.

■ Cyclomatic Complexity > 10 = code needs refactoring. $MC = E - N + 2P$.

★ Best of luck for your placements! Revise SOLID principles, SDLC models, and design patterns daily. ✓